

Parameter Server vs. All-Reduce en entrenamiento distribuido de redes neuronales: diseño y evaluación experimental en O.N.O.

Lorenzo Minervino
Facultad de Ingeniería, Universidad de Buenos Aires
Aprendizaje Profundo
Junio 2026

Resumen—El entrenamiento distribuido de redes neuronales se apoya en dos familias de algoritmos para agregar gradientes: el *Parameter Server* (PS), con servidores centrales que concentran y actualizan los pesos, y *All-Reduce* (AR), donde los workers reducen gradientes entre sí sin servidor central. Presentamos O.N.O., un sistema de entrenamiento distribuido propio en el que todos los nodos son idénticos y un orquestador les asigna el rol (worker o servidor) en tiempo de ejecución. Sobre O.N.O. comparamos AR y PS (este último sincrónico, con barrera) en MNIST, bajo un criterio de *batch global fijo* (necesario porque en O.N.O. el *batch_size* es por worker), midiendo convergencia, throughput y escalabilidad. La evidencia observada indica que, en este entorno homogéneo sobre una sola máquina, All-Reduce iguala a Parameter Server en convergencia y lo supera en throughput y tiempo hasta accuracy, porque el servidor central de PS introduce un cuello de serialización. PS, en cambio, escala con pendiente más pronunciada al crecer los nodos; sus demás ventajas estructurales (sharding, heterogeneidad, asincronía) no se ejercitan en este entorno. Las conclusiones están acotadas a un clúster simulado sobre una única máquina de ocho núcleos.

Index Terms—entrenamiento distribuido, deep learning, Parameter Server, All-Reduce, agregación de gradientes, escalabilidad.

I. INTRODUCCIÓN

A medida que crecen los modelos y los datos, el entrenamiento de redes neuronales se vuelve inviable en un único proceso y se distribuye sobre varios workers que calculan gradientes en paralelo sobre particiones de los datos (SGD distribuido con paralelismo de datos) [1]. El cuello de botella deja de ser el cómputo y pasa a ser la *comunicación*: cómo se agregan los gradientes y cómo se sincronizan los pesos. Dos enfoques dominan. El *Parameter Server* mantiene los pesos en uno o más servidores; los workers envían gradientes y reciben parámetros actualizados [2]. *All-Reduce* elimina el servidor: los workers intercambian y promedian gradientes entre sí, típicamente en anillo [3], [4], y cada uno aplica el mismo update.

Pregunta de investigación. ¿Cuándo conviene cada enfoque, por qué, y qué compromisos aparecen entre convergencia, throughput, escalabilidad y sincronización? No buscamos el estado del arte en visión, sino caracterizar el *sistema* de entrenamiento.

Contribución. (i) Describimos O.N.O., un sistema distribuido propio donde los roles se asignan en runtime y que implementa AR y PS; (ii) aportamos una comparación

experimental honesta AR vs. PS sobre MNIST con un criterio de comparación justa (*batch global fijo*); y (iii) destilamos una matriz de decisión sobre cuándo preferir cada enfoque, acotada a las condiciones evaluadas.

II. FUNDAMENTOS Y DISEÑO

II-A. SGD distribuido con paralelismo de datos

Con W workers, cada uno procesa un mini-batch local de tamaño b y calcula un gradiente g_w . El gradiente agregado es el promedio $\bar{g} = \frac{1}{W} \sum_w g_w$, que equivale a un paso de SGD con *batch efectivo* $W \cdot b$. Promediar (y no sumar) mantiene la magnitud del gradiente, y por ende el learning rate efectivo, independiente de W . Esta distinción es central: en O.N.O. el *batch_size* es *por worker*, de modo que agregar workers a b constante cambia el batch efectivo (sección III-C).

II-B. Parameter Server

Los servidores almacenan los pesos, shardeados entre ellos cuando hay más de uno; los workers hacen *push* de gradientes y *pull* de parámetros [2]. El régimen de sincronización define el comportamiento: con *barrera* todos los workers esperan a que el server aplique el update (sincrónico, equivalente estadístico a AR); sin barrera, los workers proceden con parámetros potencialmente *stale*, ganando throughput a costa de ruido (estilo Hogwild [5]). En este trabajo usamos el régimen sincrónico (barrera con store de doble buffer), que conserva la semántica de optimización estándar.

II-C. All-Reduce

En *ring all-reduce* los W workers se ordenan en anillo y ejecutan dos fases: *scatter-reduce* (cada worker termina con un fragmento del gradiente sumado) y *all-gather* (se propaga el fragmento ya reducido) [3]. El volumen comunicado por worker es $\approx 2 \frac{W-1}{W} |g|$, independiente de W salvo factor constante, lo que da buena eficiencia de ancho de banda. No hay servidor central ni punto único de contención, pero todos avanzan al ritmo del más lento (sincronía estricta). O.N.O. promedia el gradiente reducido dividiendo por W .

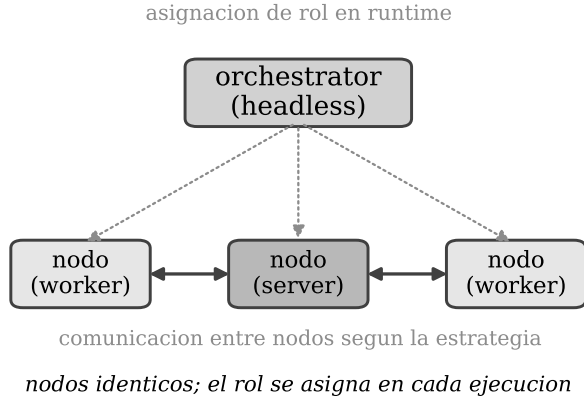


Figura 1: Arquitectura de O.N.O.: un orquestador headless asigna roles en runtime sobre nodos idénticos; según la estrategia, los nodos actúan como workers en anillo (All-Reduce) o como workers más servidores (Parameter Server).

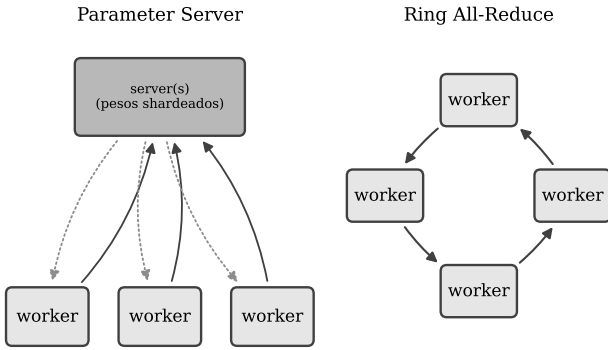


Figura 2: Parameter Server (izquierda): los workers hacen push/pull contra servidores centrales que pueden shardear los pesos. Ring All-Reduce (derecha): los workers promedian gradientes entre sí sin servidor central.

II-D. Arquitectura de O.N.O.

En O.N.O. todos los nodos son idénticos y agnósticos al rol. Un orquestador headless se conecta a ellos y, mediante una configuración de runtime, fija el rol de cada uno (worker o servidor): los roles *no* están fijados de antemano, sino que se asignan en cada ejecución. El sistema separa el motor de aprendizaje, la comunicación (protocolo y serialización) y las estrategias de agregación en componentes independientes, y está implementado en Rust. Figura 1 resume la arquitectura y figura 2 contrasta PS con ring all-reduce.

III. METODOLOGÍA EXPERIMENTAL

III-A. Datasets y modelos

Usamos **MNIST** [6] (60 000/10 000, 28×28 en escala de grises, 10 clases, normalizado a $[0, 1]$) como benchmark base de correctitud, convergencia y escalabilidad. Cuadro I resume

Cuadro I: Datasets utilizados.

Dataset	Train	Test	Forma	Clases
MNIST	60 000	10 000	28×28	10

Cuadro II: Modelos y tamaño aproximado.

Modelo	Arquitectura	Parámetros
nielsen	Conv20@5-Pool-FC100-FC10	289 630
mlp_small	FC128-FC10	101 770
mlp_medium	FC512-FC512-FC10	669 706
mlp_large	FC1024-FC1024-FC10	1 863 690

los datasets y cuadro II las arquitecturas. El modelo principal es *nielsen* (una CNN pequeña con softmax y entropía cruzada). Para aislar la presión de comunicación definimos MLPs escalables (*mlp_small/medium/large*) cuyo objetivo no es la accuracy sino variar el tamaño de gradiente.

III-B. Estrategias y configuraciones

Comparamos dos estrategias distribuidas: All-Reduce (todos los nodos son workers) y Parameter Server (barrera con store de doble buffer). Las topologías distribuidas se detallan en cuadro III. Para PS con N nodos usamos $N - 1$ workers y 1 servidor. Evaluamos $N \in \{3, 5\}$.

III-C. Criterio de comparación justa

En O.N.O. *batch_size* es *por worker*; el batch efectivo es $W \cdot b$. Por lo tanto comparamos de dos formas complementarias. (1) **Batch global fijo**: fijamos $W \cdot b$ y derivamos b según el número de workers, lo que mantiene la semántica de optimización constante y es el criterio válido para *convergencia*. (2) **Batch por worker fijo**: b constante, que mide *throughput bruto* pero cambia el batch efectivo y por ende no es justo para convergencia. Reportamos ambos y somos explícitos sobre qué pregunta responde cada uno.

III-D. Métricas y entorno

Medimos pérdida de entrenamiento por época, accuracy de test (argmax sobre los 10 000 de test, siempre el set completo), tiempo total, samples/s, speedup y eficiencia de escalado. El motor no calcula *test loss*, por lo que esa columna queda como NA. Entorno: una única máquina de 8 núcleos y 7.6 GB de RAM; el clúster se *simula* con contenedores Docker (un contenedor por nodo, red bridge, puertos publicados). Al aumentar el número de nodos los ocho núcleos físicos se saturan: medimos escalado *lógico* bajo recursos compartidos, no escalado multi-máquina (ver sección V-A). Optimizador SGD; semilla fija para reproducibilidad.

IV. RESULTADOS

IV-A. Correctitud y convergencia

Ambas estrategias entrenan correctamente: con la receta de referencia alcanzan en MNIST una accuracy de test cercana al óptimo del dataset, con PS convergiendo marginalmente más bajo en pérdida. Figura 3 muestra que las curvas de pérdida distribuidas descienden de forma casi solapada, lo que confirma

Cuadro III: Configuraciones distribuidas evaluadas.

Estr.	Nodos	W	S	sync/store	Batch ef.
AR	3	3	0	—/—	192
AR	5	5	0	—/—	240
PS	3	2	1	barrier/blocking	128
PS	5	4	1	barrier/blocking	240

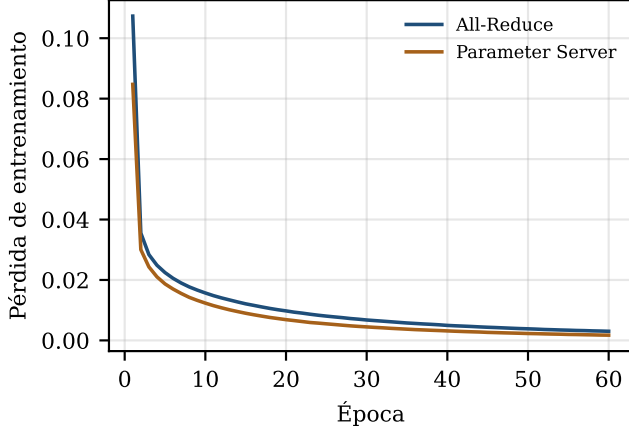


Figura 3: Convergencia en MNIST (nielsen, 3 nodos). All-Reduce y Parameter Server convergen de forma casi idéntica; PS queda apenas por debajo.

que ambos esquemas de agregación implementan la misma semántica de optimización.

IV-B. Comparación justa y tiempo hasta accuracy

Bajo *batch global fijo*, AR y PS convergen a una accuracy prácticamente igual (cuadro IV); el batch global mayor reduce la accuracy respecto de la receta de referencia, como era de esperar. La diferencia decisiva es el *tiempo*: AR alcanza su accuracy objetivo apreciablemente antes que PS (cuadro IV), porque el servidor central de PS actúa como punto de serialización por el que pasan todos los updates.

IV-C. Throughput y escalabilidad

A batch por worker fijo, AR sostiene mayor throughput que PS en ambos tamaños (figura 4). Sin embargo, PS *escala* con pendiente más pronunciada (figura 5): al sumar nodos su throughput crece más rápido que el de AR, de modo que casi se igualan a 5 nodos. La lectura: el servidor de PS penaliza el caso chico pero su costo fijo se amortiza al sumar workers; AR parte más alto pero su anillo sincrónico satura antes los ocho núcleos físicos.

IV-D. Presión de comunicación

Para aislar el efecto del tamaño de gradiente entrenamos MLPs de tamaño creciente (cuadro II) a 3 nodos, con batch por worker fijo. Figura 6 (eje logarítmico) muestra que el throughput se desploma al crecer el modelo: la comunicación, no el cómputo, domina. Comparando estrategias, All-Reduce sostiene mayor throughput que Parameter Server tanto con el

Cuadro IV: Resultados principales en MNIST/nielsen (batch global fijo).

Estr.	Nodos	Acc. (%)	Tiempo (s)	samp./s
AR	3	94.75	589	4 077
PS	3	94.69	797	3 012
AR	5	94.70	493	4 865
PS	5	94.75	501	4 792

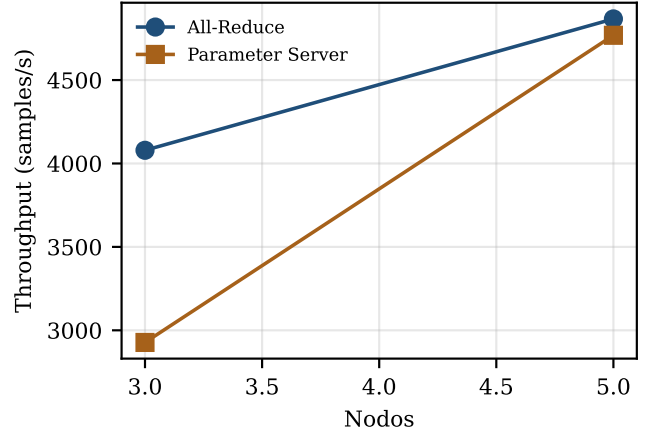


Figura 4: Throughput (samples/s) vs. nodos, batch por worker fijo. AR domina pero PS recorta distancia al crecer los nodos.

modelo chico como con el grande, y empatan en el tamaño intermedio. La presión de comunicación hunde el throughput absoluto pero no invierte la ventaja de All-Reduce: su anillo, sin servidor central, sigue siendo más rápido aun cuando el gradiente denso domina el costo. La compresión de gradientes, esparsa o cuantizada, es una mitigación conocida para esta presión [7], [8], fuera del alcance de este trabajo.

V. DISCUSIÓN

Cuándo conviene All-Reduce. La evidencia favorece AR en el escenario evaluado: clúster homogéneo, red rápida (loopback), entrenamiento sincrónico y gradientes densos. Iguala a PS en convergencia y entrega mayor throughput y menor tiempo hasta accuracy, sin un servidor central que congestione. Es la opción por defecto cuando no hay heterogeneidad que explotar.

Cuándo conviene Parameter Server. La evidencia favorece a PS en un único aspecto del entorno evaluado: escala con mayor pendiente al sumar nodos, amortizando su costo fijo de servidor. Su ventaja teórica más fuerte —el sharding de parámetros— sería decisiva para modelos que no entran en un solo nodo, pero el mayor que evaluamos sí completó en All-Reduce, así que no se ejercitó (figura 6). Lo mismo con sus demás ventajas (heterogeneidad, stragglers, asincronía), que quedan como trabajo futuro.

V-A. Limitaciones

El clúster es simulado en una sola máquina de 8 núcleos: el escalado es lógico, no físico, y la red es loopback (sin latencia ni

Cuadro V: Matriz de decisión All-Reduce vs. Parameter Server (acotada al entorno evaluado).

Condición	Recomendada	Justificación / evidencia
Clúster homogéneo, buena red, sincronía, gradientes densos, evitar servidor central	All-Reduce	Sin punto central de contención; comunicación por worker $\approx$ independiente de W . <i>Evidencia:</i> iguala a PS en accuracy con mayor throughput y menor tiempo hasta accuracy.
Modelos que no entran en un nodo, mayor escala, sharding de parámetros, desacoplar workers/servers	Parameter Server	<i>Evidencia:</i> escala con mayor pendiente al sumar nodos. El sharding sería decisivo para modelos que no entran en un nodo, no ejercitado aquí (el mayor evaluado completó en All-Reduce).

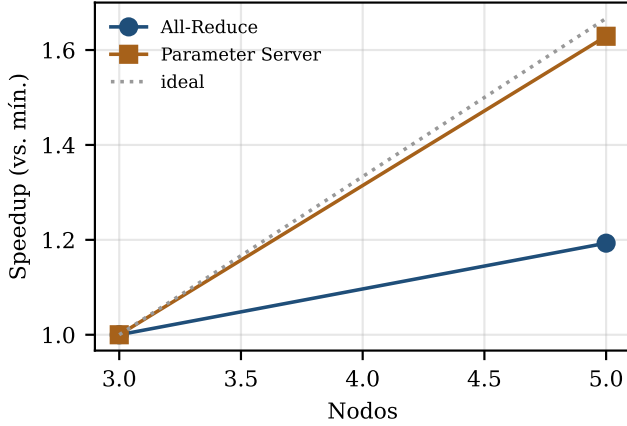


Figura 5: Speedup de throughput de 3 a 5 nodos (referencia: la propia config de 3 nodos). PS escala con mayor pendiente que AR desde una base más baja.

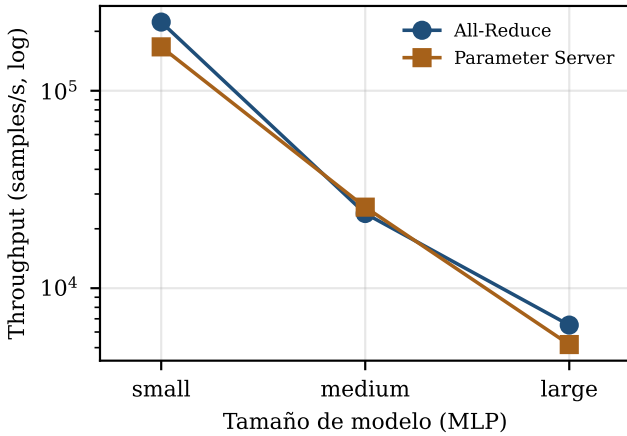


Figura 6: Presión de comunicación: throughput (log) vs. tamaño de MLP a 3 nodos. All-Reduce sostiene mayor throughput que Parameter Server en los extremos (modelo chico y grande) y empatan en el intermedio; el throughput absoluto se desploma al crecer el gradiente.

ancho de banda realistas). No medimos *test loss* (limitación del motor) y nos restringimos al régimen sincrónico. No evaluamos stragglers, heterogeneidad de nodos ni datasets más allá de MNIST, que quedan como extensiones reproducibles. O.N.O. es un sistema académico: *no* compete con frameworks industriales

como Horovod [4] o BytePS [9]; los usamos solo como marco conceptual.

VI. CONCLUSIÓN

Respondiendo a la pregunta de investigación, en el entorno evaluado (homogéneo, sincrónico y sobre una sola máquina) *All-Reduce es preferible*: iguala a Parameter Server en convergencia a batch global fijo y lo supera en throughput y tiempo hasta accuracy, porque el servidor central de PS introduce un cuello de serialización. PS recorta distancia al escalar (mayor pendiente desde una base más baja), pero no superó a All-Reduce en throughput ni siquiera con el modelo más grande; sería la elección natural solo si se explotaran sus ventajas estructurales (heterogeneidad, asincronía, sharding de modelos que no entran en un nodo), que aquí no se ejercitaron. O.N.O. aporta evidencia de que un sistema distribuido implementado desde cero, con asignación de roles en runtime, permite contrastar ambas familias bajo un criterio de comparación justa. Como trabajo futuro: red con latencia realista, stragglers controlados y la evaluación de modelos y datasets de mayor escala.

REFERENCIAS

- [1] T. Ben-Nun and T. Hoefler, “Demystifying parallel and distributed deep learning: An in-depth concurrency analysis,” *ACM Computing Surveys*, vol. 52, no. 4, pp. 1–43, 2019.
- [2] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, “Scaling distributed machine learning with the parameter server,” in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, 2014, pp. 583–598.
- [3] A. Gibiansky, “Bringing HPC techniques to deep learning,” Baidu Research, technical report, 2017, ring all-reduce for distributed gradient aggregation.
- [4] A. Sergeev and M. Del Balso, “Horovod: Fast and easy distributed deep learning in TensorFlow,” *arXiv preprint arXiv:1802.05799*, 2018.
- [5] F. Niu, B. Recht, C. Ré, and S. J. Wright, “Hogwild!: A lock-free approach to parallelizing stochastic gradient descent,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 693–701.
- [6] Y. LeCun, C. Cortes, and C. J. Burges, “The MNIST database of handwritten digits,” <http://yann.lecun.com/exdb/mnist/>, 1998.
- [7] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, “Deep gradient compression: Reducing the communication bandwidth for distributed training,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [8] A. F. Aji and K. Heafield, “Sparse communication for distributed gradient descent,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 440–445.
- [9] Y. Jiang, Y. Zhu, C. Lan, B. Yi, Y. Cui, and C. Guo, “A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 463–479.